B2.9 Data transfer

A data payload is included in Read transactions, Write transactions, Combined Write transactions, Atomic transactions, and Snoop responses with data. This section defines the data alignment rules and the data bytes that are accessed for different combinations of address, transaction size, and memory type.

In this section, unless explicitly stated otherwise, a reference to a Write transaction includes both the individual Write transaction and the corresponding Combined Write transaction.

B2.9.1 Data size

The Size field in a packet is used, in combination with other fields, to determine the number of bytes transferred in a message.

Table B2.15 shows the Size field value encodings. Snoop transactions do not include a Size field. All snoop data transfers are 64-byte.

Table B2.15: Size field value encodings

Size[2:0]	Bytes
`0b000`	1
`0b001`	2
`0b010`	4
`0b011`	8
`0b100`	16
`0b101`	32
`0b110`	64
`0b111`	Reserved

Each data message in a multi-request transaction has an effective Size of 64-bytes.

B2.9.2 Bytes access in memory

The MemAttr[1] bit field determines if the memory type is Device or Normal. See B2.8.3 *Memory Attributes*. The bytes that are accessed are determined by the memory type as follows:

Normal memory Transactions with a Normal memory type access the number of bytes defined by the Size field. Data access is from the `Aligned_Address`, that is, the transaction address rounded down to the nearest Size boundary and ends at the byte before the next Size boundary.
This is calculated as:
`Start_Address` = Addr field value.
`Number_Bytes` = 2^{Size} field value.
`INT(x)` = Rounded down integer value of x.
`Aligned_Address = (INT(Start_Address / Number_Bytes)) x Number_Bytes`.
The bytes accessed are from `(Aligned_Address)` to `(Aligned_Address + Number_Bytes) - 1`.

Device memory Transactions with a Device memory type access the number of bytes from the transaction address up to the byte before the next Size boundary.
The bytes accessed are from `(Start_Address)` to `(Aligned_Address +`

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

`Number_Bytes) - 1.`
For Write transactions to Device locations, BE bits must only be 1 for the bytes that are accessed. See B2.9.3 *Byte Enables*.

B2.9.3 Byte Enables

Byte Enables (BE) are used alongside Write transactions and Snoop responses with Data.

In the following section, unless explicitly stated otherwise, a reference to a Write transaction includes both the individual Write transaction and the corresponding Combined Write transaction.

In WriteData and Snoop responses, a data BE value of 0 must set the associated data byte value to 0.

In CompData and DataSepResp messages, the BE bits are inapplicable and can take any value.

B2.9.3.1 Write transactions

For all Write transactions, BE bits that are not within the data window, specified by Addr and Size, must be 0.

When BE is 1 in Write transactions, this indicates that the associated data byte is valid and must be updated in memory or cache.

When BE is 0, this indicates that the associated data byte is not valid and must not be updated in memory or cache.

A Requester must set all BE values to 0 in CopyBackWriteData_I and WriteDataCancel packets.

During data transfers in the following write transactions:

- All BE bits must be 1, except when write data is a CopyBackWriteData_I or WriteDataCancel packet:
  - WriteNoSnpFull
  - WriteNoSnpDef
  - WriteBackFull
  - WriteCleanFull
  - WriteEvictFull
  - WriteUniqueFull
  - WriteUniqueFullStash
- Any combination of BE bits is permitted, including all ones and all zeros:
  - WriteBackPtl
  - WriteUniquePtl
  - WriteUniquePtlStash

For a WriteNoSnpPtl transaction, the following rules apply:

- For a transaction to Normal memory, any combination of BE bits can be 1 during the data transfers. This includes all ones and all zeros.
- For a transaction to Device memory, BE bits must only be 1 for bytes at or above the address specified in the transaction. Any combination of BE bits can be 1 that meets this requirement.
  This includes all ones and all zeros.

B2.9.3.2 Atomic transactions

For Atomic transactions, all BE bits within the data window must be 1.

For Atomic transactions, BE bits that are not within the data window, as specified below by Addr and Size, must be 0:

- If Addr is aligned to Size, the Data window is `[Addr:(Addr+Size-1)]`.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- If Addr is not aligned to Size, the Data window is `[(Addr-Size/2):(Addr+Size/2-1)]`.

B2.9.3.3 Snoop transactions

For Snoop responses with data that use the SnpRespData or SnpRespDataFwded opcode, all BE bits must be 1.

For Snoop responses with data that use the SnpRespDataPtl opcode, any combination of BE bits can be 1 alongside the data transfers. This includes all ones and all zeros.

B2.9.4 Data packetization

For each transaction that involves data, the data bytes can be transferred in multiple packets.

The number of packets required is determined by:

- Transaction Size
- Data_Width

Use of Limited Data Elision can reduce the number of packets that need to be transported. See B2.9.5 *Limited Data Elision*.

The maximum number of bytes transferred in each packet is determined by:

- Data_Width

The CHI protocol supports the following data bus widths:

- 128-bit
- 256-bit
- 512-bit

The *Data Identifier* (DataID) and *Critical Chunk Identifier* (CCID) fields are used to identify data packets within a transaction.

A transaction size of up to 16-byte is always contained in a single packet. The DataID can be calculated for each packet based on the data bus width:

- 128-bit: bits [5:4] of the effective memory address of any byte in the packet
- 256-bit: bit [5] of the effective memory address of any byte in the packet, concatenated with `0b0`
- 512-bit: always `0b00`

For different data bus widths, Table B2.16 shows the relationship between the DataID field and the bytes that are contained within the packet.

Table B2.16: DataID and the bytes within a packet for different data widths

DataID	Data Width		
	128-bit	256-bit	512-bit
`0b00`	Data[127:0]	Data[255:0]	Data[511:0]
`0b01`	Data[255:128]	Reserved	Reserved
`0b10`	Data[383:256]	Data[511:256]	Reserved
`0b11`	Data[511:384]	Reserved	Reserved

Within a data packet, all bytes are located at their natural byte positions. This is true even if fewer data bytes are transferred than the width of the data bus.

The number of data packets used for transactions to Device memory is independent of the address of the transaction. The number of data packets required is determined only by the Size field and the data bus width.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

When a DAT message is split into multiple packets:

- The fields that must be consistent are:
  - TgtID
  - SrcID
  - TxnID
  - HomeNID
  - PBHA
  - Opcode
  - Resp
  - FwdState
  - DataPull
  - DataSource
  - DBID, when the field is applicable, or inapplicable with a fixed value
  - CCID
  - TagOp
  - CAH
  - MECID
  - MismatchedMECID
  - CacheLineID
- The fields that are expected to be consistent, but can vary, are:
  - QoS
  - RespErr
  - DBID, when the field is inapplicable and can take any value
  - TraceTag
  - CBusy
- The fields that are expected to vary are:
  - Data
  - DataID
  - Tag
  - RSVDC
  - BE
  - DataCheck
  - Poison
  - NumDat
  - Replicate

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B2.9.5 Limited Data Elision

Limited Data Elision enables an optional reduction in the number of data packets transferred on the DAT channel when either:

- Some or all of the Data field values are 0
- Subsequent data packets contain replicated Data field values

Each data packet transferred has an indication of number of data packets elided from the data message.

The use of Limited Data Elision does not affect the transaction structure.

B2.9.5.1 Approach

The NumDat field is used in a DAT packet to indicate how many of the sequentially numbered DAT packets are elided. For any elided packet, the Replicate field is used to indicate its field values.

If a packet is elided, the Data field can either take the same value as the sent packet, Replicate = `0b1`, or zero, Replicate = `0b0`. See Figure B2.40, Figure B2.41, and Figure B2.42 for examples of how messages could be communicated using Limited Data Elision.

The maximum number of packets that can be elided is one less than the maximum required to transport a full cache line of data. The number of packets that can be elided could be less depending on the request data size.

The possible values for NumDat and Replicate for different data bus widths and requested data sizes is shown in Table B2.17.

Table B2.17: Possible NumDat and Replicate values

Data Width	Request size	NumDat[1:0]	Replicate
512	Any	`0b00`	`0b0`[a]
256	512	`0b00`	`0b0`[a]
		`0b01`	`0b0` or `0b1`
	<=256	`0b00`	`0b0`[a]
128	512	`0b00`	`0b0`[a]
		`0b01`	`0b0` or `0b1`
		`0b10`	`0b0` or `0b1`
		`0b11`	`0b0` or `0b1`
	256	`0b00`	`0b0`[a]
		`0b01`	`0b0` or `0b1`
	<=128	`0b00`	`0b0`[a]

[a] Inapplicable and must be zero.

When the transaction size is less than 64B:

- Packets with DataID `0b00` and `0b10` are permitted to have a non-zero NumDat value.
- Packets with DataID `0b01` and `0b11` must have NumDat = `0b00`.

The issuing of elided data from an interface can be controlled using the **BROADCASTLIMELISION** signal.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B2.9.5.2 Examples

Figure B2.40 details a number of example DAT channel messages at the packet level before and after the inclusion of Limited Data Elision for a 128 bit data bus.

Note

Examples where it is not possible to elide any packets from a data message are also included in Figure B2.40.

Original			
Packet 3 DataID=11	Packet 2 DataID=10	Packet 1 DataID=01	Packet 0 DataID=00
0	0	0	0
A	A	A	A
0	0	0	A
0	A	0	0
C	C	B	A
A	A	0	0
		0	A
		A	0
0	A		
A	0		

With Limited Data Elision			
Packet 3 DataID=11	Packet 2 DataID=10	Packet 1 DataID=01	Packet 0 DataID=00
Not sent	Not sent	Not sent	0 NumDat=3 Replicate=0 or 1
Not sent	Not sent	Not sent	A NumDat=3 Replicate=1
Not sent	Not sent	Not sent	A NumDat=3 Replicate=0
Not sent	A NumDat=3 Replicate=0	Not sent	Not sent
Not sent	C NumDat=1 Replicate=1	B NumDat=0 Replicate=0[a]	A NumDat=0 Replicate=0[a]
A NumDat=2 Replicate=0	A NumDat=0 Replicate=0[a]	Not sent	Not sent
		Not sent	A NumDat=1 Replicate=0
		A NumDat=0 Replicate=0[a]	0 NumDat=0 Replicate=0[a]
Not sent	A NumDat=1 Replicate=0		
A NumDat=0 Replicate=0[a]	0 NumDat=0 Replicate=0[a]		

[a] Inapplicable and must be zero.

Figure B2.40: 128-bit Data bus width examples

Figure B2.41 details a number of example DAT channel messages at the packet level before and after the inclusion of Limited Data Elision for a 256 bit data bus.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Original		With Limited Data Elision	
Packet 1 DataID=10	Packet 0 DataID=00	Packet 1 DataID=10	Packet 0 DataID=00
0	0	Not sent	0 NumDat=1 Replicate=0 or 1
0	0	0 NumDat=1 Replicate=0 or 1	Not sent
A	A	Not sent	A NumDat=1 Replicate=1
A	A	A NumDat=1 Replicate=1	Not sent
A	0	A NumDat=1 Replicate=0	Not sent
0	A	Not sent	A NumDat=1 Replicate=0

Figure B2.41: 256-bit Data bus width examples

B2.9.5.3 Applicability and restrictions

There are various circumstances to be aware of when using Limited Data Elision, outlined in this section:

- B2.9.5.3.1 *MTE*
- B2.9.5.3.2 *BE*
- B2.9.5.3.3 *NumDat and Replicate values for CopyBackWriteData_I, WriteDataCancel, or NDERR*

B2.9.5.3.1 MTE

When using MTE, a DAT packet can be elided when it has the same TagOp, TU, and Tag values as the packet being sent.

B2.9.5.3.2 BE

To support Limited Data Elision, the packet being sent must have BE bits as all ones or all zeros.

B2.9.5.3.3 NumDat and Replicate values for CopyBackWriteData_I, WriteDataCancel, or NDERR

When Limited Data Elision is used to elide packets from CopyBackWriteData_I or WriteDataCancel messages, or data messages with RespErr = NDERR, the maximum number of packets must always be elided for that message.

When Limited Data Elision is used to elide packets from CopyBackWriteData_I and WriteDataCancel messages, Replicate must be zero.

Figure B2.42 details some CopyBackWriteData_I, WriteDataCancel and RespErr = NDERR messages, before and after the inclusion of Limited Data Elision for a 128-bit data bus.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

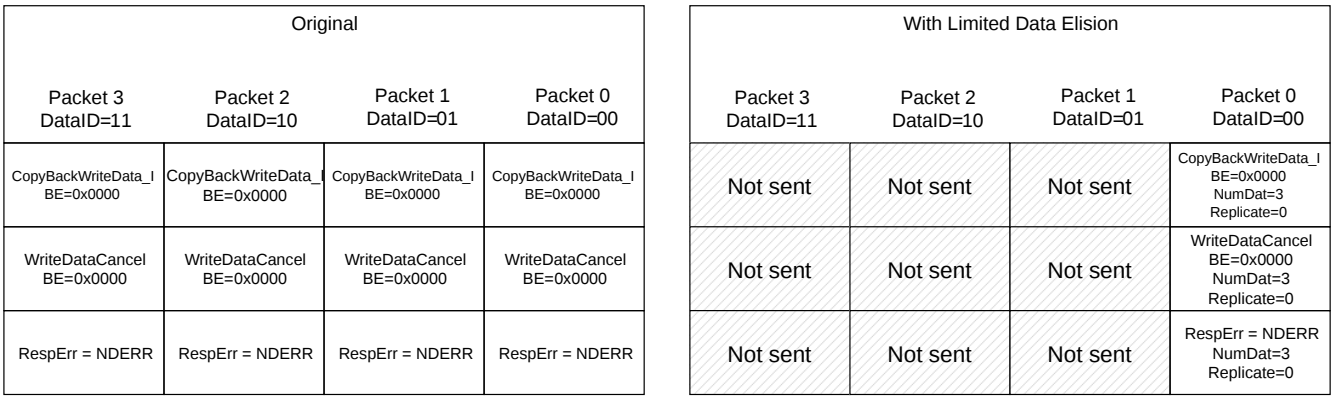

Figure B2.42: CopyBackWriteData_I, WriteDataCancel, or NDERR Data bus width example

B2.9.6 Size, Address, and Data alignment in Atomic transactions

This section describes the data size and alignment requirements for Atomic transactions. It contains the following sections:

- B2.9.6.1 *Size*
- B2.9.6.2 *Address and Data alignment*
- B2.9.6.3 *Endianness*

B2.9.6.1 Size

The Size field of the packet specifies the total data size of the Atomic transaction.

For the AtomicCompare transaction, the data size is the sum of the Compare and Swap data values.

Table B2.18 shows the permitted data sizes, and the relationship between inbound and outbound valid data size for each Atomic transaction type. The size of the data value returned in response to an AtomicCompare transaction is half the number of bytes specified in the Size field in the associated Request packet.

Table B2.18: Atomic transaction outbound and inbound data sizes

Atomic transaction	Outbound (bytes)	Inbound
AtomicStore	1, 2, 4, or 8	-
AtomicLoad	1, 2, 4, or 8	Same as outbound
AtomicSwap	1, 2, 4, or 8	Same as outbound
AtomicCompare	2, 4, 8, 16, or 32	Half size of outbound

B2.9.6.2 Address and Data alignment

In the AtomicStore, AtomicLoad, and AtomicSwap transactions:

- The byte address is aligned to the outbound data size.
- The position of data bytes in the NonCopyBackWriteData or any CompData packet matches the endianness of the operation, as specified in the Endian field of the request.
- The big-endian data is byte invariant.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

The write data associated with an AtomicCompare transaction is provided in the same way as a transaction that is aligned to the outbound data size.

In the AtomicCompare transaction:

- The byte address must be aligned in the Data packet to the inbound data size, which is equivalent to half the outbound data size.

The two data values in an AtomicCompare transaction are placed in the data field in the following manner:

- The Compare and Swap data values are concatenated and the resulting data payload is aligned in the Data packet to the outbound data size.
- The Compare data is always at the addressed byte location.
- The Swap data is always in the remaining half of the valid data.

For any given Compare data address, the Swap data address can be determined by inverting bit[n] in the Compare data address where:

n = $\log_2$(Compare data size in bytes)

B2.9.6.2.1 Alignment example

Figure B2.43 shows the examples of data placement with different addresses and different Data sizes.

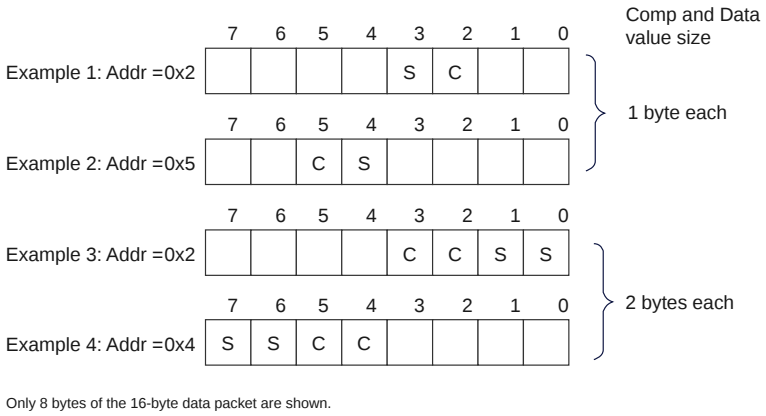

Figure B2.43: Data value packing for AtomicCompare transaction

In Figure B2.43, Example 1 shows the addressed byte location is `0x2` and the total size of data is 2 bytes. In this case, the Compare and Swap data must be placed in an address location aligned to a 2-byte boundary that includes the addressed location, that is, addresses `0x2` to `0x3`. Compare data is placed in location `0x2` and Swap data is placed in location `0x3`.

> **Note**
>
> The address of the Swap data can be determined by inverting bit[0] of the Compare data address. Bit[0] is inverted because the size of the Compare data and the size of the Swap data is 1 byte.

In Figure B2.43, Example 3 shows the addressed location is `0x2` and the total size of data is 4 bytes. In this case, the Compare and Swap data must be placed in an address location aligned to a 4-byte boundary that includes the addressed location, that is, addresses `0x0` to `0x3`. Compare data is placed in location `0x2` and Swap data is placed in location `0x0`.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Note

The address of the Swap data can be determined by inverting bit[1] of the Compare data address. Bit[1] is inverted because the size of the Compare data and the size of the Swap data is 2 bytes.

B2.9.6.3 Endianness

The data on which an atomic operation executes can be in either little-endian or big-endian format. For arithmetic operations, such as ADD, MAX, and MIN, the component performing the operation needs to know the format of the data.

The endian format of the data is defined by the Endian bit in the Atomic transaction Request packet. See B13.10.30 *Endian*.

B2.9.7 Critical Chunk Identifier

The CCID field is used to identify the data bytes that are the most critical in the transaction request.

The CCID field must match the value of Addr[5:4] of the original request. Transactions which contain multiple data packets must use the same CCID value for all data packets.

When read data or write data is reordered by the interconnect, the CCID field permits quick identification of the most critical bytes within a transaction by comparing the CCID value with the DataID value. When the two values match, the data bytes being transferred are the critical bytes.

The bits to match is dependent on the data bus width:

- For a data bus width of 128 bits, the CCID and DataID bits must match for the critical chunk.
- For a data bus width of 256 bits only the most significant CCID and DataID bits must match for the critical chunk.

B2.9.8 Critical Chunk First Wrap order

The Sender of Data is permitted, but not required, to send individual Data packets of a transaction in critical chunk first wrap order.

The interface property, CCF_Wrap_Order defines the capabilities of a Sender, and the guarantees provided by the Receiver:

- CCF_Wrap_Order at the Sender:

  True The Sender signals the Data packets can be sent in critical chunk first wrap order.

  False The Sender signals the Data packets cannot be sent in critical chunk first wrap order.

- CCF_Wrap_Order at the interconnect:

  True The interconnect signals the Data packets are guaranteed to maintain the order that transactions are received.

  False The interconnect signals the Data packets are not guaranteed to maintain the order that transactions are received.

- CCF_Wrap_Order at the Receiver that is not an interconnect:

  True The Receiver requires the Data packets to be received in critical chunk first wrap order.

  False The Receiver does not require the Data packets to be received in critical chunk first wrap order.

If some components in the system do not support sending Data packets in critical chunk first wrap order, the receiver of Data must not rely on Data being received in critical chunk first wrap order.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Note

At design time, the CCF_Wrap_Order parameter can help a component to identify if Data packets need to be sent in critical chunk first wrap order. For example, if the component is aware to be connected to an out-of-order interconnect, its Data packet path could be simplified by not returning the Data packets in critical chunk first wrap order.

If the interconnect has the CCF_Wrap_Order property set to True, a component interfacing to that interconnect can send Data packets in critical chunk first wrap order, if capable. The Receiver can subsequently make use of possible latency optimization, due to receiving the critical chunk first.

B2.9.9 Data Beat ordering

The reordering of data packets within a transaction is permitted when passing across an interconnect. However, the original source of data packets is permitted, but not required, to provide the packets in a critical chunk first, wrap order. See B2.9.8 *Critical Chunk First Wrap order*.

Note

Critical chunk first wrap order ensures that interfacing to protocols that do not support data reordering, such as AXI, can be done in the most efficient manner when an ordered interconnect is used.

Wrap order is defined as follows:

`Start_Address` = Addr

`Number_Bytes` = 2^{Size}

`INT(x)` = Rounded down integer value of x

`Aligned_Address = (INT(Start_Address / Number_Bytes))` x `Number_Bytes`

`Lower_Wrap_Boundary = Aligned_Address`

`Upper_Wrap_Boundary = Aligned_Address` + `Number_Bytes` - 1

To maintain wrap order, the order must be:

1. The first data packet must correspond to the data bytes specified by the Start_Address of the transaction.
2. Subsequent packets must correspond to incrementing byte addresses up to the Upper_Wrap_Boundary.
3. Subsequent packets must correspond to the Lower_Wrap_Boundary.
4. Subsequent packets must correspond to incrementing byte addresses up to the Start_Address.

Note

Some of the steps to maintain wrap order could overlap and not be required if the required bytes are included in a previous step.

B2.9.10 Data transfer examples

This section gives a number of examples of the data transfer requirements.

In most of the examples, the size of the transaction is 64 bytes and the data bus width is 128 bits. This requires 4 data packets for each transaction.

In the following examples, the accompanying text highlights some interesting aspects. The examples are not intended to describe all aspects.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

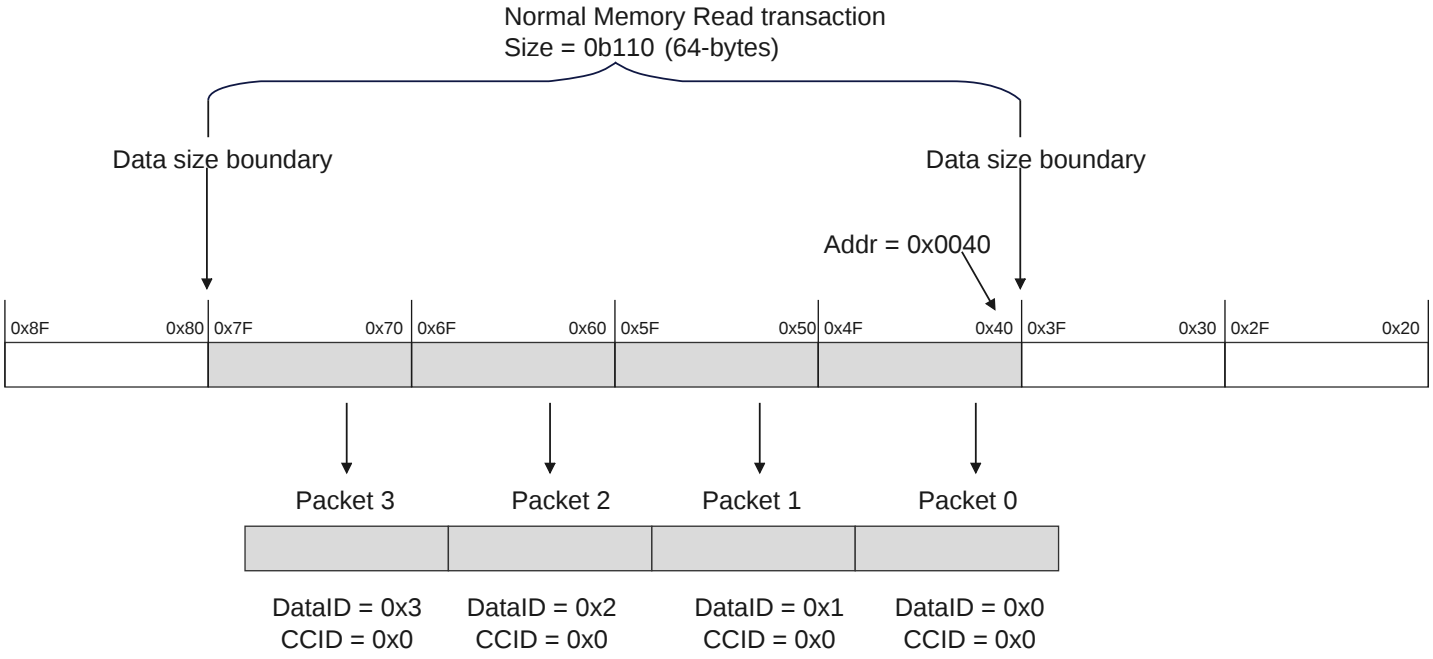

Figure B2.44: Normal memory 64-byte Read transaction from an aligned address

In Figure B2.44:

- The order of the data packets, as indicated by Packet 0, Packet 1, Packet 2, and Packet 3, is such that they follow wrap order.
- The DataID changes for each packet, while the CCID field remains constant.
- The packet containing the data bytes specified by the address of the transaction has the same value for the CCID and DataID fields.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

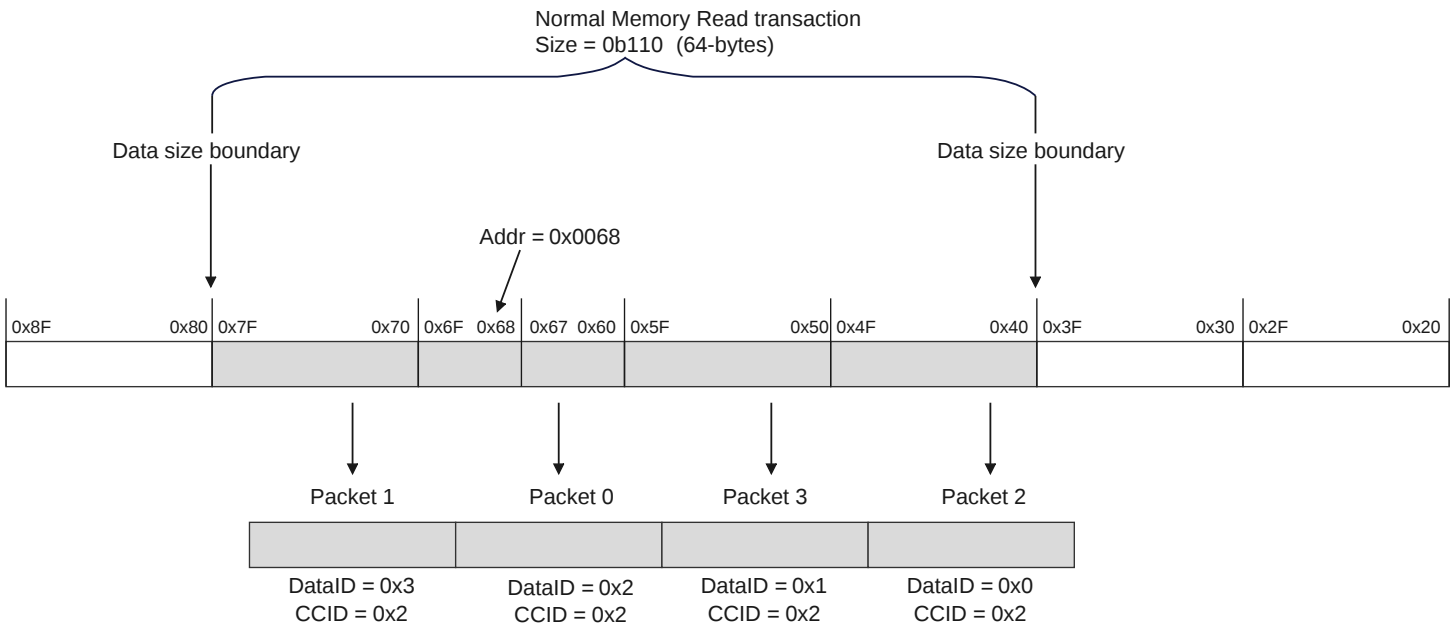

Figure B2.45: Normal memory 64-byte Read transaction from an unaligned address

In Figure B2.45:

- The order of the data packets, as indicated by Packet 0, Packet 1, Packet 2, and Packet 3, is such that they follow wrap order.
- The DataID changes for each packet, while the CCID field remains constant.
- The packet containing the data bytes specified by the address of the transaction has the same value for the CCID and DataID fields.

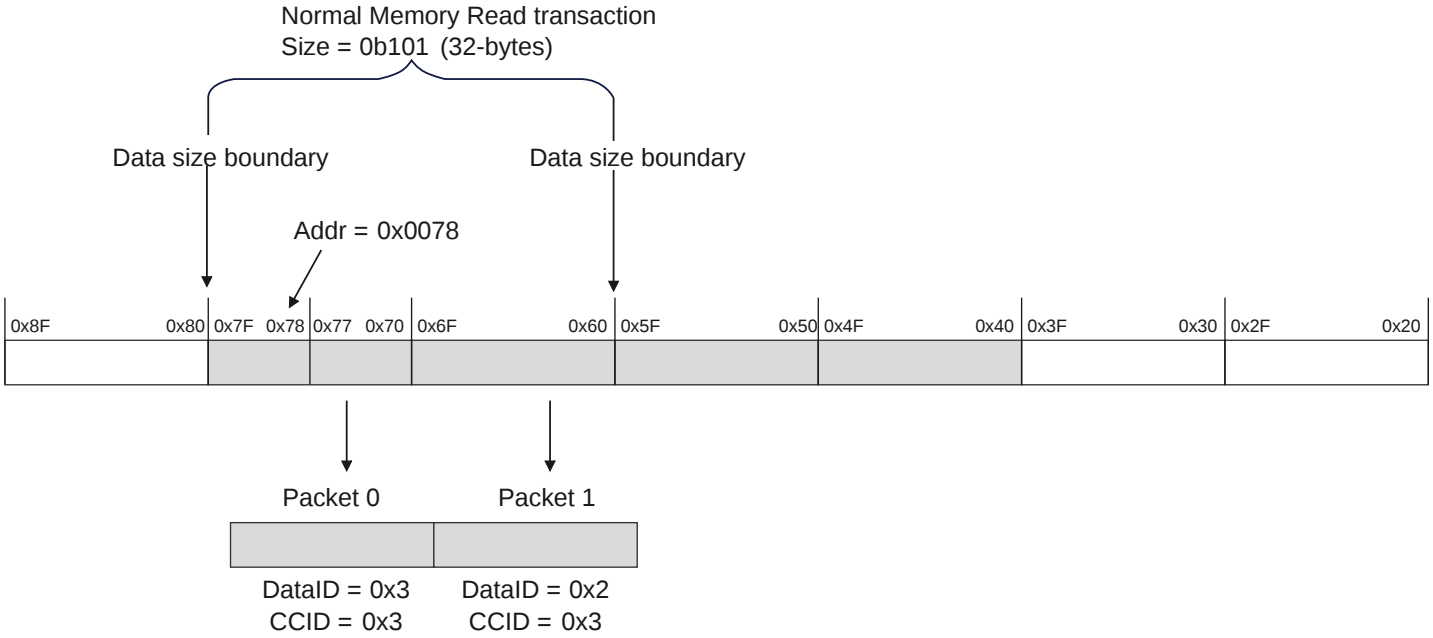

Figure B2.46: Normal memory 32-byte Read transaction from an unaligned address

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

In Figure B2.46:

- The size of the transaction is 32-byte and the data bus width is 128-bit, resulting in 2 data packets.
- The order of the data packets, as indicated by Packet 0 and Packet 1, is such that they follow wrap order.

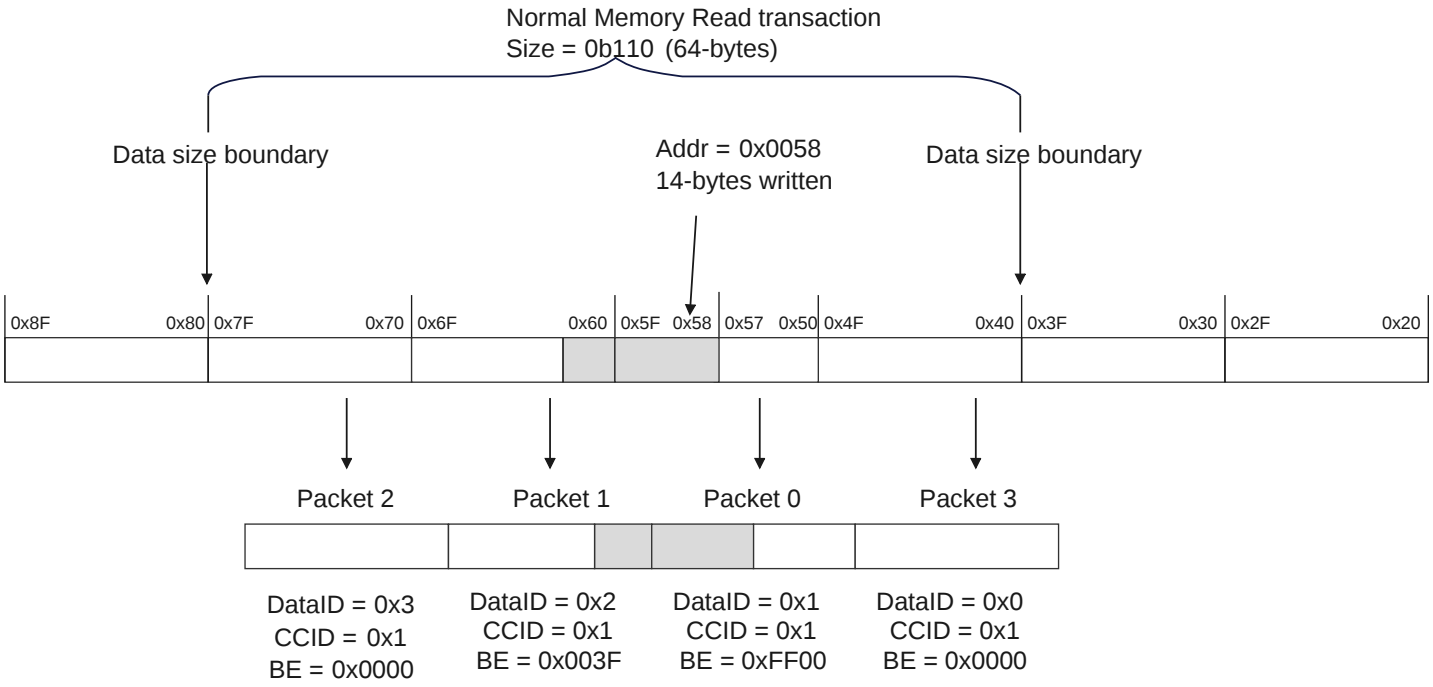

Figure B2.47: Normal memory 14-byte Read transaction from an unaligned address

In Figure B2.47:

- The order of the data packets, as indicated by Packet 0, Packet 1, Packet 2, and Packet 3, is such that they follow wrap order.
- The DataID changes for each packet, while the CCID field remains constant.
- The packet containing the data bytes specified by the address of the transaction has the same value for the CCID and DataID fields.
- Fourteen consecutive bytes in memory are written, as indicated by the BE bits. However, other combinations of BE bits are permitted. See B2.9.3 *Byte Enables*.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

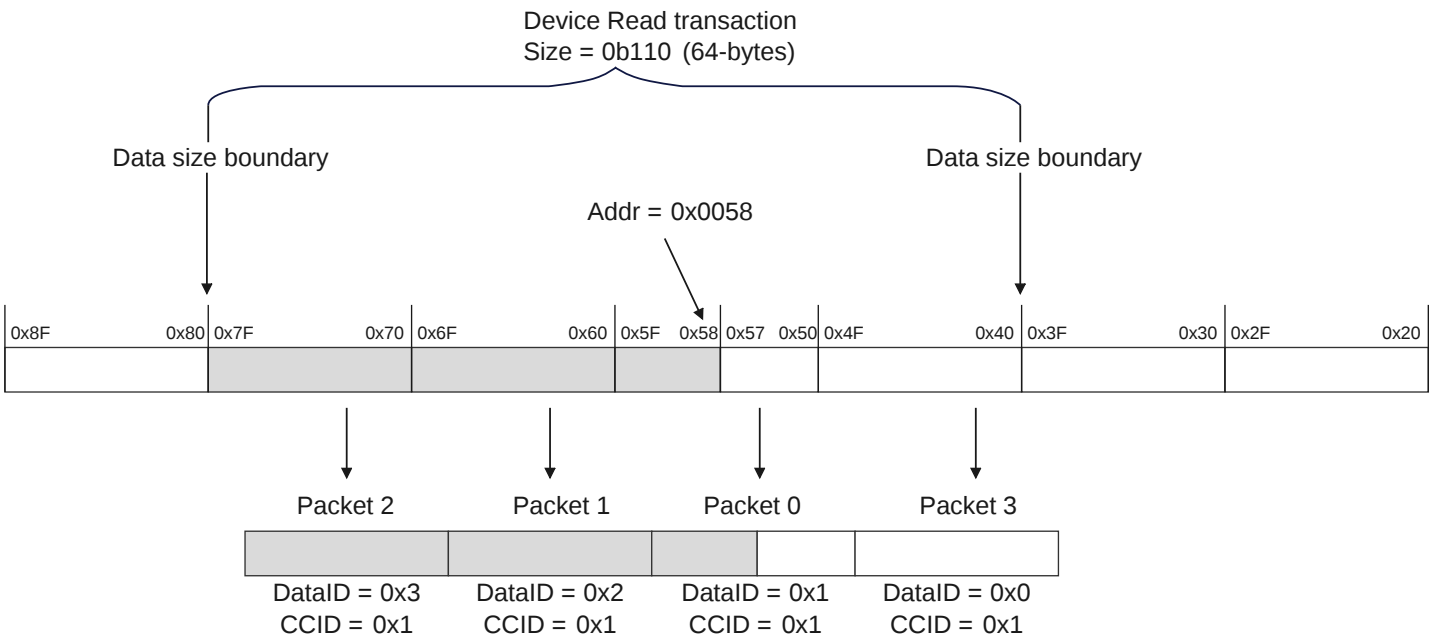

Figure B2.48: Device Read transaction from an unaligned address

In Figure B2.48:

- The shaded area indicates the valid bytes in the transaction. The valid bytes extend from the transaction address up to the next Size boundary.
- The transaction includes the transfer of a packet that contains no valid data.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

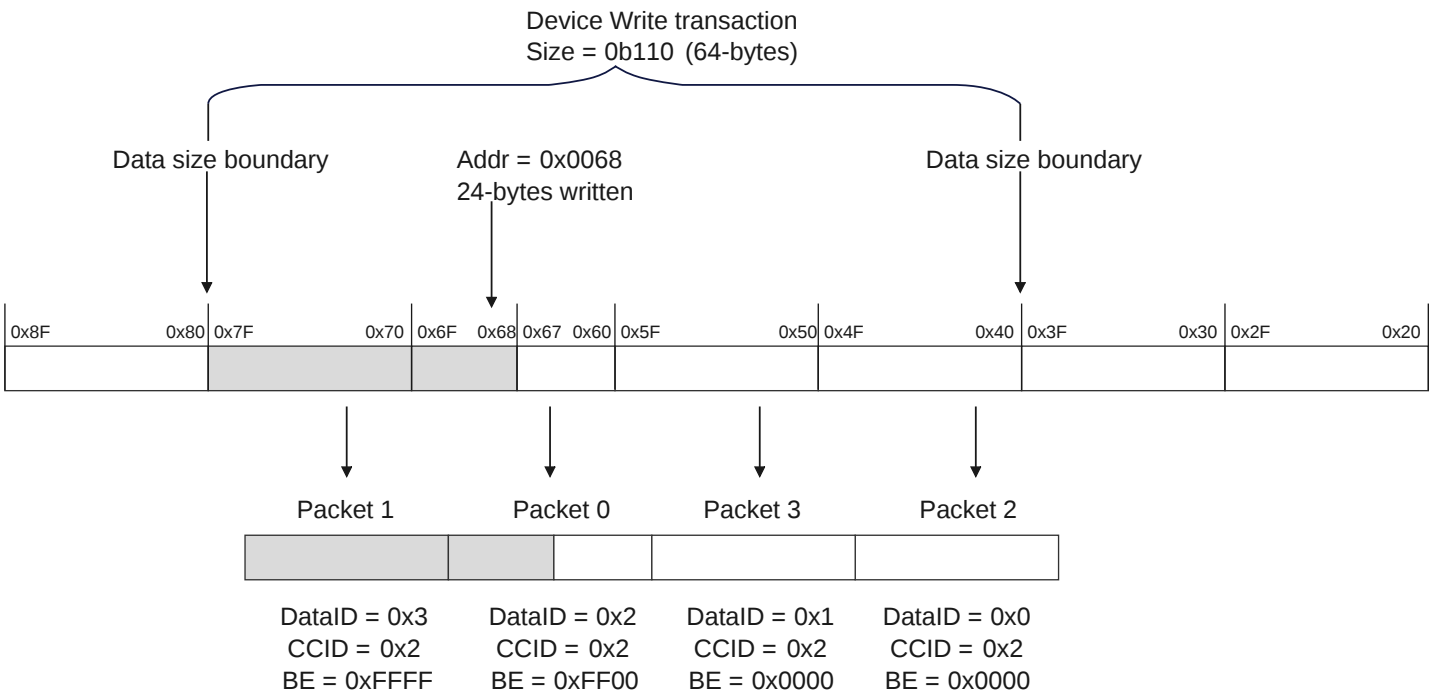

Figure B2.49: Device Write transaction to an unaligned address

In Figure B2.49:

- BE bits are only permitted to be 1 for the bytes from the transaction address up to the next Size boundary. It is not required that all BE bits meeting this criteria are 1.
- BE bits for bytes below the start address must be 0.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential